

Data Compression Overview

Video script with synchronized blackboard figures

This is the lecture notes for the YouTube video, with every figure placed exactly where it should be drawn. Figures that build up over the course of the video (the master tree, the Shannon-limit wall) appear more than once, each time showing only what should be on the board *at that point* – draw onto the same space rather than starting over.

Part 1 — Lossless vs. lossy

This video gives you a bird’s-eye view of data compression algorithms and techniques. We won’t go deep into any single algorithm here. However, this high-level view is critical to understanding the overall field of data compression. More importantly, it helps you see where the algorithms in this beautiful YouTube course fit within the bigger picture. During the video, you’ll be creating a large tree diagram. This shows the hierarchy of compression algorithms and techniques. A leaf of the tree will have an actual algorithm, whereas a non-leaf will have the categories that algorithms belong to. Let’s start!

Every data compression algorithm falls into one of two broad categories: lossless and lossy.

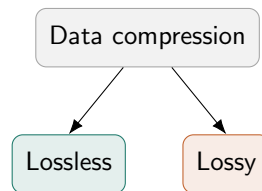


Figure 1: The master tree, stage 1. Just the root and the two top-level branches – this same tree keeps growing for the rest of the video.

Let’s understand exactly what separates these two categories. For that, let’s look at a simple block diagram.

Imagine you have some original data. We’ll call it x . This data goes into a compressor, also called an encoder. What comes out the other side is a smaller, compressed version of that data. We’ll call it c . Later, when you want your data back, you take that compressed data c and feed it into a decompressor, also called a decoder. What comes out is the recovered data. We’ll call it $\bar{x}$.

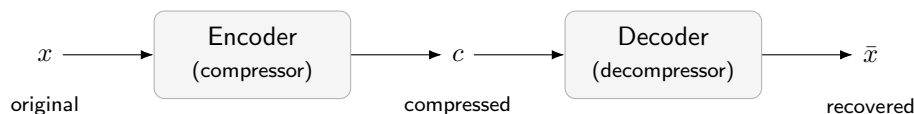


Figure 2: The encoder/decoder pipeline. Every other figure in this video about lossless vs. lossy is really just zooming in on what happens to $\bar{x}$ at the very end of this diagram.

The entire difference between lossless and lossy compression comes down to one question. How does $\bar{x}$

compare to the original x . In lossless compression, $\bar{x}$ is always identical to x . No matter what the input was, you are guaranteed to get back the exact original data, bit for bit. In lossy compression, $\bar{x}$ is not identical to x . Some information was discarded during the encoding process. That information is gone for good. So what you get back is only an approximation of the original. It is not an exact copy.

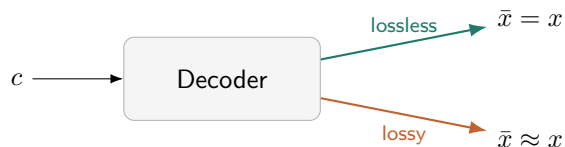


Figure 3: The fork that defines the whole field: exact recovery (teal, lossless) or approximate recovery (orange, lossy). This annotation sits directly on top of fig. 2’s decoder output.

It makes sense to use lossy compression when the data is multimedia. That means audio, video, or images. Losing some data in exchange for better compression might not even be visible to the naked eye. It might not even be audible to human ears. But in every other case, you need the data to come back exactly as it was. Think of the textual records of government employees. In those cases, you resort to lossless compression instead.

Part 2 — The Shannon limit

Before we divide lossless compression into its different categories, let’s first discuss one fundamental idea. It is one of the greatest theoretical results in data compression. It is called the Shannon limit.

Claude Shannon showed that every source of data has a compression limit. This result is known as Shannon’s source coding theorem. It tells you the maximum amount by which the data can be compressed without losing any information. The limit depends on the amount of information, or uncertainty, in the source. This quantity is called entropy.

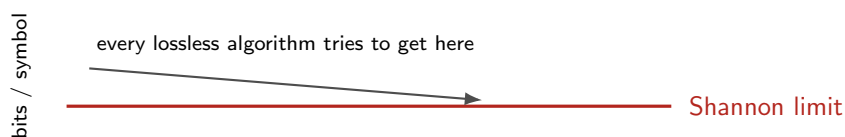


Figure 4: The Shannon limit, stage 1: a wall that no lossless algorithm can cross, with one generic arrow approaching it. No specific algorithms yet – those get added to this same wall much later, in fig. 14.

I have separate videos on entropy and the Shannon limit, so I won’t discuss them in detail here. The key idea is simple. No lossless compression algorithm, no matter how clever, can achieve an average compressed size below this limit.

So although the algorithms we are about to study are very different, they all have the same goal. They try to compress data as close to the Shannon limit as possible while remaining fast and memory efficient.

Part 3 — The five families of lossless compression

With that in mind, lossless compression can be divided into five categories. Number one, entropy coding. Number two, dictionary or LZ family. Number three, transform-based compression. Number four, context modeling. And number five, multimedia, formats that combine the first four families, specialized for video, images, and audio.

Let us first briefly describe each of these categories. Then we will zoom into each one and build more tree diagrams.

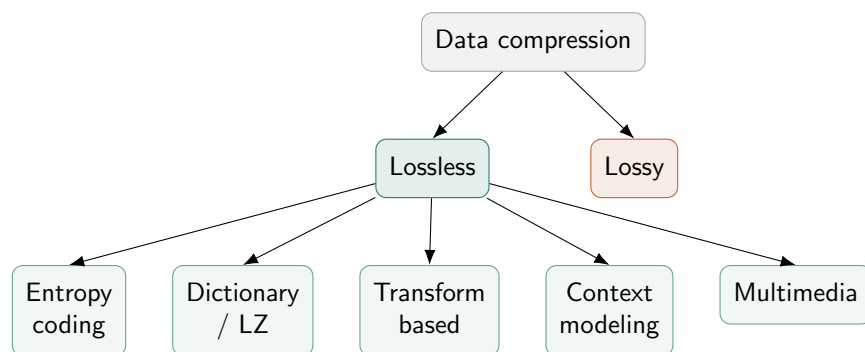


Figure 5: The master tree, stage 2: Lossless gains its five children. The first four are algorithmic families; the fifth, multimedia, is built by combining them, specialized for video, images, and audio – we’ll see exactly how in [Part 8](#). Lossy is still left empty on purpose – it is not covered in this video.

Let’s start with entropy coding. Entropy coding is a family of techniques. These techniques assign shorter codes to more probable symbols. They assign longer codes to less probable ones. For example, in English text, the letter *e* appears much more often than the letter *x*. So an entropy coder would assign a shorter code to *e*. It would assign a longer code to *x*.

‘e’ (common) → code: 0
‘x’ (rare) → code: 1011001

Figure 6: The intuition behind every entropy coder, before any formal definitions: common symbols get short codes, rare symbols get long ones.

This idea is actually quite intuitive. Symbols that occur more frequently should take fewer bits to represent. Symbols that occur less frequently can afford to use more bits. As a result, the average number of bits per symbol goes down. This is how entropy coding approaches the Shannon limit we just talked about.

Part 4 — Entropy coding: prefix vs. non-prefix

If we zoom into entropy coding, we find two further groups of algorithms. These are prefix coding and non-prefix coding.

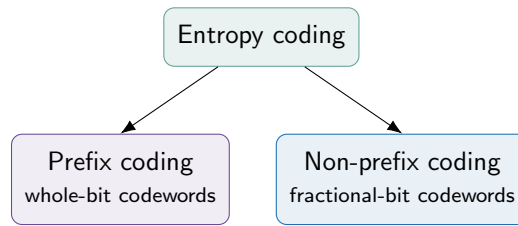


Figure 7: The master tree, stage 3: Entropy Coding gains its two children. Violet = prefix coding, blue = non-prefix coding – these two colors are reused for the rest of the entropy-coding section.

A prefix code is a type of code where no codeword is the prefix of any other codeword. This means you can read a stream of bits from left to right. And you can always tell exactly where one codeword ends. There is no ambiguity.

For example, consider this mapping. A is encoded as 0. B is encoded as 10. C is encoded as 110. D is encoded as 111. This is a prefix code. None of these codewords starts with another complete codeword.

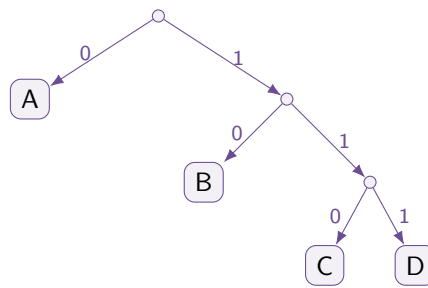


Figure 8: A valid prefix code as a binary tree: $A = 0$, $B = 10$, $C = 110$, $D = 111$. Every letter is a *leaf* – none of them sits on the path to another letter, which is exactly what “prefix-free” means.

Now consider this mapping instead. A is encoded as 0. B is encoded as 01. C is encoded as 10. D is encoded as 11. This is not a prefix code. The codeword for A , which is 0, is a prefix of the codeword for B , which is 01. This creates ambiguity when decoding.

$$A = 0 \quad B = 01 \quad C = 10 \quad D = 11$$

0 1 (the next two bits received by the decoder)

read as A? (done)

 or wait, it's actually B = 01?

Figure 9: Why this mapping is ambiguous: after seeing the single bit 0, the decoder cannot tell whether to stop (it’s A) or keep reading (it might be the start of $B = 01$). This is the one “error” figure in the section, marked in red.

Shannon coding, Shannon-Fano coding, and Huffman coding are all prefix codes. Shannon coding takes each symbol’s ideal code length, $-\log_2 p$, and rounds it up to the nearest integer to get that symbol’s codeword length. Shannon-Fano works top-down, repeatedly splitting the sorted symbol list into two balanced halves. Huffman coding builds the optimal prefix code for a given probability distribution

using a greedy tree-construction algorithm. Huffman is optimal among all prefix codes for a memoryless source.

There’s a fourth prefix code worth naming here: Rice coding. It’s still entropy coding in the same sense as the other three – common values get short codewords, rare values get long ones – but it gets its probabilities a different way. Huffman, Shannon, and Shannon-Fano all estimate the probability of every symbol by measuring the data and building an explicit table. Rice coding instead assumes the data already follows one specific, very common shape: a geometric distribution, where small values are likely and large values get exponentially rarer. That shape happens to match prediction residuals from audio and image compression almost perfectly. To encode a value, Rice coding splits it into a quotient and a remainder, writes the quotient in unary, a string of 1s ended by a 0, so smaller quotients really do get shorter codewords, and the remainder in a fixed number of bits set by a single tunable parameter k . Golomb coding, Rice coding’s more general parent, is provably the optimal prefix code for a geometric source; Rice coding is just the special, division-free case where that parameter is a power of two. That’s the real difference from the rest of this section: not that it ignores probability, but that it gets probability from a formula instead of a frequency count, which is exactly why it’s cheap enough to refit block by block rather than requiring a full pass over the data up front the way Huffman does. We’ll see it again with FLAC in [Part 8](#).

Prefix codes are constrained by the Kraft McMillan inequality. One important consequence of this constraint is that every codeword must have an integer number of bits. In our earlier example, A was assigned a code of length 1. B was assigned a code of length 2. C and D were both assigned codes of length 3. These lengths are all integers. We cannot assign D a code of length 2.4 in Huffman coding.

Because of this restriction, Huffman coding cannot always match the exact entropy of the source. So it cannot always reach the Shannon limit. Arithmetic coding is different. It is a non-prefix entropy coding method. It can assign each symbol a code of fractional length. So in our example, a symbol like D could actually be assigned a code of length 2.4.

Symbol	Probability	Huffman (prefix)	Arithmetic (non-prefix)
A	0.43	1 bit	1.1 bits
B	0.25	2 bits	1.9 bits
C	0.14	3 bits	2.7 bits
D	0.18	3 bits	2.4 bits
Average length (bits/symbol)		1.89 bits	1.76 bits

Figure 10: Why Huffman can’t always reach the entropy limit: every Huffman codeword length is a whole number (violet column), while a non-prefix coder like Arithmetic coding can use fractional lengths (blue column) – a length of 2.4 bits, marked in red, is simply impossible for D under Huffman coding. The probability column (derived from each symbol’s ideal code length, $p \approx 2^{-\text{length}}$) lets us actually weigh these lengths: averaged over how often each symbol occurs, Huffman lands at 1.89 bits/symbol, while Arithmetic coding lands at 1.76 bits/symbol (green) – a real, measurable gap, not just a per-symbol curiosity.

Because of this, arithmetic coding can often get very close to the Shannon limit. In many cases it gets closer than Huffman coding does. This means it can offer a noticeably better compression ratio. But arithmetic coding is also noticeably slower. It relies on fractional arithmetic, including division. To solve this problem, researchers developed range coding. Range coding uses only integer arithmetic. It avoids division entirely. Range coding gives you exactly the same compression ratio as arithmetic

coding. But it runs faster.

More recently, many streaming applications have started using entropy coding. One example is Zstandard, which is used by companies like Netflix and YouTube. For these applications, even range coding is not fast enough. Users could experience an annoying delay during streaming. To solve this, researchers developed a family of algorithms called ANS. These are faster than both arithmetic coding and range coding. But that extra speed comes at a small cost. ANS gives you a slightly lower compression ratio in exchange, very close to arithmetic and range coding but not quite as good. Within this family, rANS, which stands for range ANS, is best suited for parallel computing. Most streaming compression tools instead use tANS, which stands for table ANS. tANS is the fastest of them all, since both encoding and decoding reduce to simple table lookups.

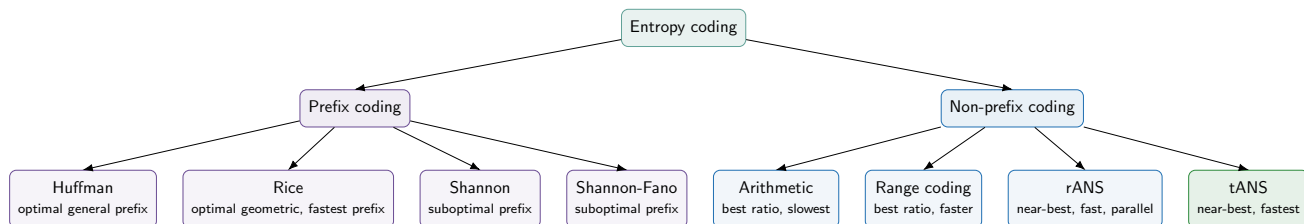


Figure 11: The entropy-coding branch, fully grown. Green marks tANS as the fastest of the eight – pure table lookups, no arithmetic at all.

So here is how these seven entropy coders actually compare. Huffman is the quickest and simplest to build, but it gives up the most compression ratio, since it’s stuck with whole numbers of bits. Shannon and Shannon-Fano are also prefix codes, but neither is guaranteed optimal even among prefix codes – Huffman always does at least as well. Arithmetic coding and range coding sit at the other end. They give you the best possible compression ratio, right up against the Shannon limit. Between the two, range coding is faster than arithmetic coding, since it avoids division. rANS and tANS sit in between on ratio, very close to arithmetic and range coding but not quite as good, and they are the fastest of the group when it comes to actually running the encoder and decoder, with tANS being the fastest of all since it relies purely on table lookups.

Let’s pin the comparison down exactly, with two ranked tables.

Rank	Algorithm(s)	Why
1	Arithmetic, Range	Tied. Both use fractional-length code-words and get essentially as close to the Shannon limit as possible. Range coding is just an integer reformulation of arithmetic coding – same ratio.
2	rANS, tANS	Tied. Also non-prefix, fractional-effective-length coders, so they get very close to Arithmetic/Range – but the table-driven, finite-precision approach gives up a small, deliberate amount of ratio in exchange for speed.
3	Huffman	Optimal <i>among prefix codes</i> for any distribution. Locked into whole-bit code-words, so it can't match the non-prefix coders above it, but better than any other prefix coder in general.
4	Rice	Optimal <i>among prefix codes</i> specifically for geometric distributions. On geometric data, matches Huffman's ratio; on non-geometric data, performs worse. Its advantage is speed and simplicity, not ratio.
5	Shannon, Shannon-Fano	Tied. Both are prefix codes like Huffman, but neither is guaranteed optimal even among prefix codes – Huffman always does at least as well as either of them.

Figure 12: All entropy coders we've named, ranked by compression ratio, best to worst. Colors match the prefix/non-prefix split from figs. 7 and 11.

Now let's rank the same seven algorithms a completely different way: by speed.

Rank	Algorithm(s)	Why
1	tANS	Fastest of all. Encoding and decoding reduce to pure table lookups – no arithmetic operations at all.
2	Rice	Extremely fast. No probability table needs to be built – the distribution is assumed from a single parameter k , so encoding/decoding is just simple bit manipulation: unary for the quotient, fixed bits for the remainder.
3	Huffman, Shannon, Shannon-Fano	Tied. All three are prefix codes – encode/decode is a simple lookup of a fixed bit pattern per symbol, structurally similar and similarly fast regardless of which one assigns the codewords. (Building the table is another matter – Huffman’s table construction is fast, Shannon/Shannon-Fano are faster to build but worse on ratio.)
4	rANS	Fast, but each symbol still requires a bit of integer arithmetic to update the state, so it’s a step behind the pure-lookup tANS.
5	Range	Uses integer arithmetic with renormalization but avoids division – faster than arithmetic coding, slower than the table-driven methods above.
6	Arithmetic	Slowest. Needs high-precision, renormalizing fractional arithmetic, historically including division per symbol.

Figure 13: The same entropy coders, ranked by compression speed, fastest to slowest. Notice the ranking barely resembles fig. 12’s ranking – the fastest coder (tANS) is tied for second-worst on ratio, Rice is tied for second-fastest but ranks below Huffman on general ratio, and the three slowest-to-rank-1 coders on ratio (Huffman, Shannon, Shannon-Fano) are tied for third-fastest here.

Side note: Huffman’s reputation for being “fast” usually refers to how quickly its code *table* can be built – it’s a simple, greedy tree construction. That’s a separate claim from how fast encoding and decoding run per symbol once the table already exists, which is exactly why those two ideas land in different tables above rather than getting conflated into one.

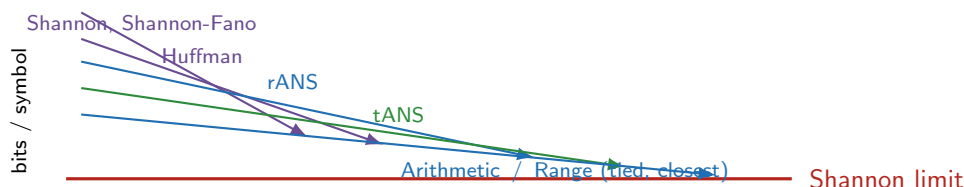


Figure 14: The Shannon limit wall, fully labeled. Arithmetic and Range coding are tied for closest to the limit – that is the entire reason they exist. rANS and tANS trade a small, deliberate gap from the limit for a large gain in speed. Huffman is further from the wall, since it is locked into whole-bit codewords. Shannon and Shannon-Fano are furthest, since they aren’t even optimal among prefix codes. Colors match fig. 11.

Part 5 — Dictionary compression: the LZ family

Next, we move to dictionary-based compression algorithms. They mostly belong to the LZ family. At a high level, we can divide them into LZ77 and LZ78.

Dictionary compressors like LZ77 replace repeated byte patterns with a short reference pointing back to where that pattern last appeared, using a sliding window of recent data to keep the reference small and efficient.

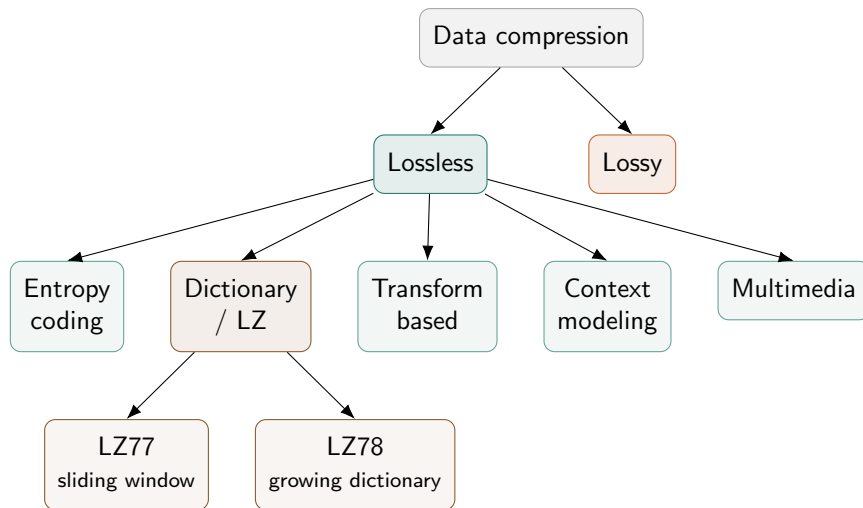


Figure 15: The master tree, stage 4: Dictionary / LZ gains its two children, LZ77 and LZ78, drawn in a new color (brown) so it’s visually distinct from the entropy-coding branch. Transform based, context modeling, and multimedia are still waiting their turn, in Parts 6 through 8.

The key difference is this. LZ77 uses a sliding window. This makes it memory constrained. LZ78 instead uses a growing dictionary.

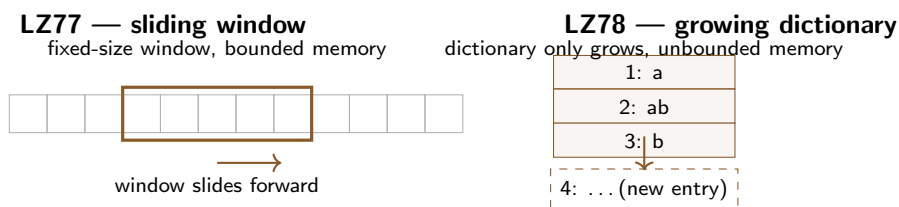


Figure 16: The core contrast between the two LZ branches. LZ77’s window (left) stays the same size and slides over the data it has already seen. LZ78’s dictionary (right) starts empty and gains a new entry every time a new substring is seen.

Most of the algorithms we cover in this course are extensions of LZ77. Now let’s see how LZ77 actually works, at a high level. It slides a fixed-size window over the data, split into two parts: a search buffer holding what’s already been seen, and a look-ahead buffer holding what’s about to be encoded. Whenever the look-ahead buffer’s next string already appears somewhere in the search buffer, LZ77 replaces it with a short reference: how far back the match starts, and how long it runs. The window then slides forward to cover that newly encoded data.

We’re skipping real details here – match-finding strategies, encoding optimizations, overlapping matches – since they deserve their own video. For now, the takeaway is simple: LZ77 swaps repeated patterns for short references, and almost every dictionary compressor in use today is built directly on this sliding-window idea.

Here’s a tiny example, shown in Figure 17. Say the look-ahead buffer’s next characters are `abc`, and that exact string already appeared earlier in the search buffer. Instead of writing `a`, `b`, `c` all over again, LZ77 just points back to that earlier copy. Because the window is small, a reference back into it is almost always shorter than the text it replaces.

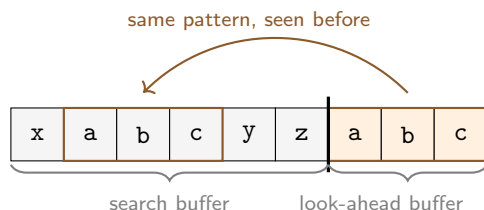


Figure 17: *

The second `abc` doesn’t get written out again – it’s replaced by a reference pointing back to the first one. We’ll unpack exactly what that reference contains in a future video.

Most real-world compressors are exactly this idea, dressed up with a particular entropy coder on top. DEFLATE is LZ77 combined with Huffman coding, and it’s the engine inside `zip`, `gzip`, and `PNG` – old, simple, and everywhere. Zstandard is LZ77 combined with `tANS`, a much newer design tuned to get close to DEFLATE’s ratio, or better, while running far faster, which is why companies like Facebook and Netflix use it for everyday compression at scale. LZMA (Lempel-Ziv-Markov chain algorithm) uses LZ77 with larger dictionaries and advanced entropy coding (range coding), achieving excellent compression ratios at the cost of slower speed. Brotli, used in web compression (`HTTP`), is a modern LZ77 variant with a static dictionary of common web strings and Huffman coding, offering better compression than `gzip` for text. Snappy, developed by Google, is an extremely fast LZ77-based compressor with modest compression, optimized for low latency in databases and real-time systems.

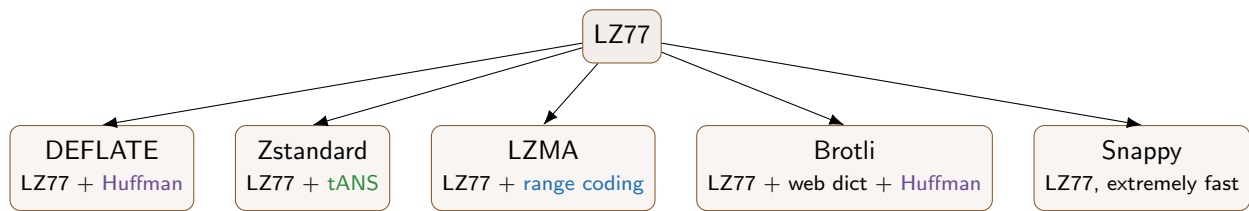


Figure 18: LZ77 in practice: five real formats, each adding an entropy coder from [Part 4](#) or a specialized optimization. The “+Huffman”, “+tANS”, and “+range coding” labels are colored to match [figs. 11](#) to [13](#) – dictionary methods don’t replace entropy coding, they hand off to it.

Just like we did with entropy coding, let’s compare these five dictionary-based formats on two dimensions: compression ratio ([Figure 19](#)) and speed ([Figure 20](#)).

Rank	Algorithm	Why
1	LZMA	Best compression ratio among dictionary methods. Uses large dictionaries (up to 4 GB) and range coding for entropy, enabling excellent ratio at the cost of speed.
2	Brotli	Excellent ratio, especially on text. Uses a large static dictionary of common web strings plus Huffman coding . Often outperforms DEFLATE on web content by around 20–26%.
3	Zstandard	Very good ratio, tunable from near-DEFLATE to near-LZMA levels. Uses LZ77 plus tANS , and at higher compression levels typically beats DEFLATE on ratio while still running at comparable or faster speed.
4	DEFLATE	Decent, general-purpose ratio. LZ77 plus Huffman coding . The original universal baseline – simple, well understood, and everywhere (zip, gzip, PNG) – though Zstandard now beats it on ratio in most benchmarks.
5	Snappy	Modest compression ratio – deliberately trades ratio for speed. Designed for databases and real-time systems where latency matters more than storage.

Figure 19: Dictionary-based formats ranked by compression ratio, best to worst. LZMA (range coding) and Brotli (Huffman with web dictionary) lead; Snappy intentionally sacrifices ratio for speed.

[Figure 19](#) ranked these formats by compression ratio. Now let’s rank the same five formats by speed, in [Figure 20](#).

Rank	Algorithm	Why
1	Snappy	Fastest of all. Minimal dictionary lookups, no entropy coding overhead. Designed specifically for maximum throughput, with compression ratio as a secondary concern.
2	Zstandard	Very fast, especially at lower compression levels. tANS is extremely efficient, and the implementation is heavily optimized. Often matches or exceeds Snappy’s decompression speed while compressing better.
3	DEFLATE	Moderate speed. Huffman coding is fast, but LZ77 match-finding can be expensive. Gzip, built on DEFLATE, has long been the default wherever “good enough” speed was all that mattered.
4	Brotli	Slower than DEFLATE. The larger dictionary and more complex match-finding cost extra CPU time. Quality over speed, especially for offline compression of web assets.
5	LZMA	Slowest of the five. Large dictionaries and complex range coding make it CPU-intensive. Designed for maximum compression, not speed – often used for software distribution, where decompression speed matters more than compression speed.

Figure 20: Dictionary-based formats ranked by compression speed, fastest to slowest. Notice the ranking is almost the reverse of Figure 19 – Snappy is fastest but worst on ratio, LZMA is best on ratio but slowest.

The key insight from Figures 19 and 20 is the same trade-off we saw with entropy coding: **you can’t have both maximum ratio and maximum speed**. Each format makes a deliberate choice about where to sit on that spectrum:

- **Snappy** prioritizes speed above all else.
- **Zstandard** offers the best overall balance – good ratio with competitive, often class-leading speed.
- **DEFLATE** isn’t really on this frontier anymore – Zstandard beats it on both ratio and speed in most benchmarks. DEFLATE survives because it’s been the universal default since the 1990s (zip, gzip, PNG), not because of where it sits on this trade-off.
- **Brotli** trades some speed for better ratio, especially on web content.
- **LZMA** goes all-in on ratio, accepting much slower compression.

Part 6 — Transform-based compression: the Burrows–Wheeler Transform

We’ve now covered two of our five lossless families: entropy coding and dictionary compression. Let’s move to the third family. Transform-based compression.

Here’s the core idea. Instead of compressing the data directly, first transform it into a completely different arrangement of the exact same symbols. An arrangement that happens to be far easier to compress. Then run a simple compressor on top of that.

The most famous example of this approach is the Burrows-Wheeler Transform, or BWT. It is the engine inside the popular bzip2 compressor.

Pause and guess: the Burrows-Wheeler Transform, by itself, does not save a single bit. Not one. So why would anyone bother using it? Take a guess, then keep watching.

Here’s how it works. Take your string, and add a unique end marker to it, say a dollar sign. Now generate every rotation of that string. Sort all of those rotations alphabetically. The BWT output is simply the last column of that sorted list.

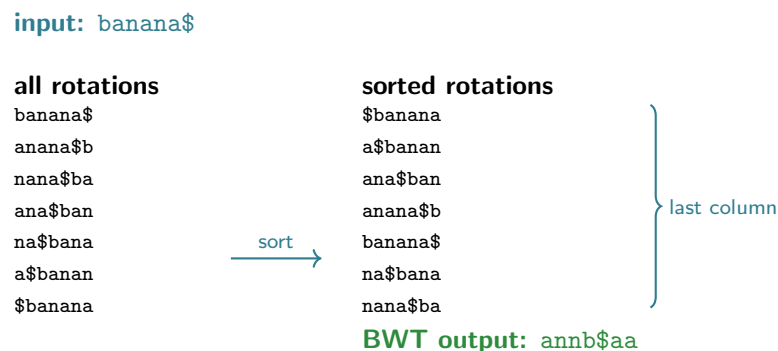


Figure 21: The Burrows-Wheeler Transform of `banana$`, worked by hand. Every rotation of the input (left) is sorted alphabetically (right), and the last letter of each sorted row, read top to bottom, is the BWT output. Notice the output has far more repeated, neighboring letters than the original string did.

So why bother? Look closely at the output: `annb$aa`. Compare that to the original: `banana$`. The letters haven’t changed, and there are exactly as many of them. But in the BWT output, identical letters are far more likely to sit right next to each other. That’s because sorting groups together every rotation that shares the same upcoming context. The BWT itself doesn’t compress anything. What it does is rearrange the data so that a much simpler compressor, typically a move-to-front transform, followed by run-length encoding, followed by entropy coding, can squeeze it far harder than it could squeeze the original. That handoff to a simple downstream compressor is exactly how bzip2 is built.

The “much simpler compressor” mentioned above is itself a second transform: the move-to-front transform, or MTF. By the same definition used for BWT, MTF qualifies as transform-based compression too – it doesn’t shrink the data by a single bit on its own, it just rearranges symbols into a far more compressible shape, then hands off to run-length and entropy coding to do the actual squeezing.

Here’s how it works. Keep a list of every distinct symbol in the data, in some starting order. For each symbol in the input, output its current position in that list, then move that symbol to the front of the

list. Symbols that keep repeating, exactly what BWT produces in long runs, end up at the front of the list almost permanently, so they keep generating small numbers, often literally zero.

Symbol	List before	Output	List after
a	\$ a b n	1	a \$ b n
n	a \$ b n	3	n a \$ b
n	n a \$ b	0	n a \$ b
b	n a \$ b	3	b n a \$
\$	b n a \$	3	\$ b n a
a	\$ b n a	3	a \$ b n
a	a \$ b n	0	a \$ b n

Figure 22: The move-to-front transform applied to the BWT output `annb$aa` from Figure 21, starting from the alphabet `$ a b n`. Each row outputs the symbol’s current position, then moves that symbol to the front. The MTF output is `1 3 0 3 3 3 0` – mostly small numbers, with three 3’s and two 0’s, exactly the kind of skewed, repetitive distribution that run-length and entropy coding exploit well.

That output, `1 3 0 3 3 3 0`, is what bzip2 hands next to run-length encoding and then entropy coding. None of the three steps, BWT, MTF, or RLE, compresses anything by itself. Each one just reshapes the data so the next stage in the pipeline has an easier job.

BWT and MTF aren’t the only transforms in this family, either. Delta encoding replaces each value with its difference from a predicted value, often just the previous one, turning smoothly changing data into a stream of small numbers; it’s the same idea behind PNG’s per-row prediction filter (Part 9). The discrete cosine transform and the discrete wavelet transform, the workhorses behind JPEG, MP3, and JPEG2000, do the same job for lossy compression: they convert a signal into frequency components so that most of the energy lands on a few large coefficients and everything else can be quantized away almost for free. DCT does this with global, whole-block waves, while wavelets use short, localized ripples that adapt to both frequency and position at once, which is what lets them keep sharp edges crisp instead of smearing them across the block. What unites all of these, BWT, MTF, delta encoding, DCT, wavelets, is the same handoff pattern: rearrange or re-express the data first, then let a simpler coder do the squeezing.

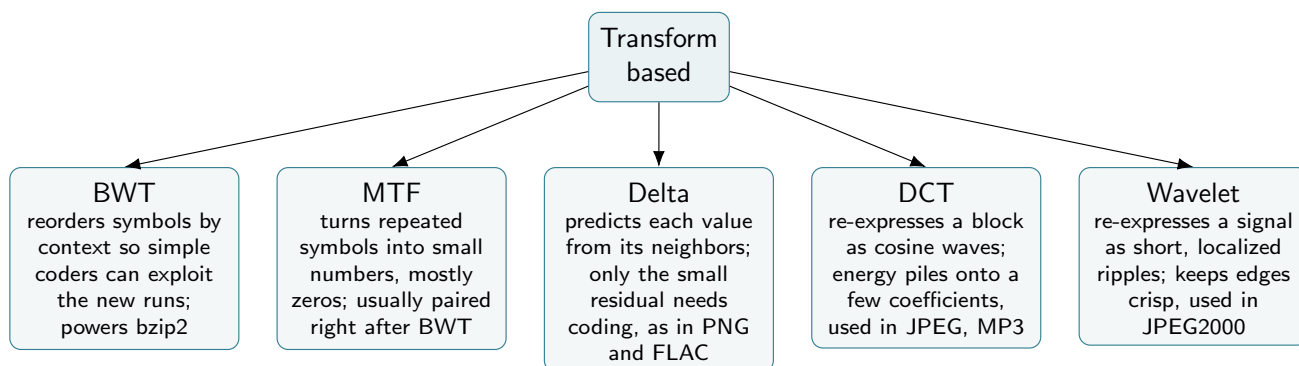


Figure 23: The transform-based branch. None of BWT, MTF, delta encoding, DCT, or the wavelet transform compete with entropy coding or LZ – each is a pre-processing step that makes whatever comes after it work better.

Part 7 — Context modeling: PPM and context mixing

That brings us to the fourth of our five lossless families: context modeling.

Entropy coding assigns a symbol's code length based on how common that symbol is, overall, across the entire source. Context modeling asks a sharper question. Forget the overall frequency. How likely is this symbol to appear *right here*, given the handful of symbols that just came before it?

Context modeling is effectively an order- n entropy model, where n is the size of the context. It estimates the probability of each symbol conditioned on the n symbols that preceded it, rather than using a single global probability for every symbol.

The flagship algorithm in this family is called PPM, which stands for Prediction by Partial Matching.

Quick check: if entropy coding already gives common symbols short codes, what extra trick is context modeling adding on top? (Answer is in the next paragraph – try to guess it first.)

Here is PPM, in plain terms. To predict the next symbol, PPM looks at the last few symbols it has already seen, that short, recent window is the *context*, the “partial match.” It then checks how often each possible symbol has followed that exact context before, and turns those counts into a probability distribution. That distribution gets handed to an entropy coder, almost always arithmetic coding, which does the actual work of writing out the bits. And if that exact context has never come up before, PPM doesn't give up: it escapes down to a shorter context, one symbol shorter at a time, until it finds one with real statistics behind it.

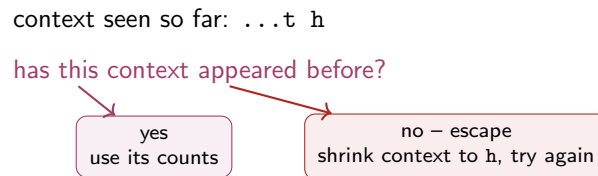


Figure 24: PPM's escape mechanism. If the current context has statistics, PPM uses them directly (left, teal-magenta). If not, it shrinks the context by one symbol and tries again (right, red), repeating until it lands on a context it has actually seen.

PPM has a more powerful, and much slower, cousin called context mixing, or CM. Instead of picking one context length and falling back when it fails, context mixing blends predictions from many different models at once, different context lengths, even entirely different kinds of models, using a weighted combination instead of an either-or choice. This family holds the records for the best compression ratios ever measured, and it's the approach behind compressors like PAQ and its descendants. The cost is speed: context mixing can run hundreds of times slower than something like Huffman coding.

Fun fact: the Hutter Prize is an ongoing competition that pays out cash for shrinking a fixed 1 GB sample of Wikipedia text as small as possible. The idea is that squeezing text harder forces you to *model* it better, and better modeling of language is a proxy for general intelligence. Every leading entry for years has been a context mixing compressor descended from PAQ – the same family we just met, pushed to its absolute limit.

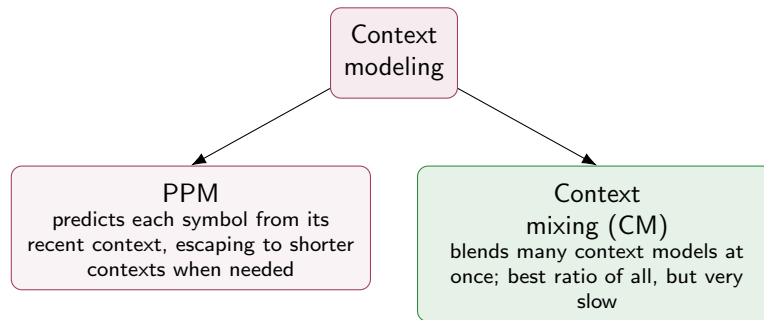


Figure 25: The context-modeling branch. Context mixing (green) sits at the extreme “best possible ratio” end of every trade-off curve we’ve drawn so far, paid for entirely in speed.

Part 8 — The fifth family: multimedia

We’ve now met four of the five algorithmic families that make up lossless compression: entropy coding, dictionary or LZ, transform based, and context modeling. Time for the fifth one we promised back in [Part 3](#): multimedia.

Earlier in this video I said multimedia, video, images, and audio, is the classic use case for *lossy* compression. That’s true most of the time. But sometimes you cannot afford to lose anything: medical scans, film archives, or a studio’s master audio recording. For those cases, engineers built dedicated lossless formats for video, images, and audio. And under the hood, every one of them is just a clever combination of the four families we already covered.

Pause and guess: FLAC is a lossless audio format. Based on everything we’ve covered so far, which lossless family or families do you think it leans on? Keep reading to check your guess.

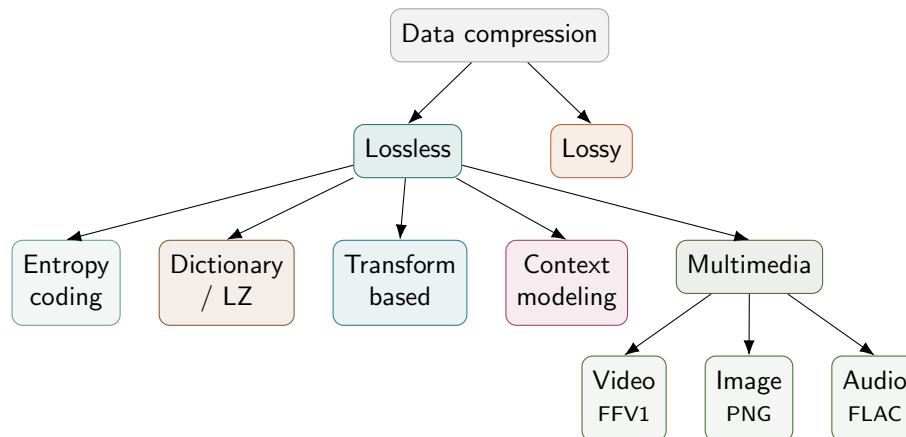


Figure 26: The master tree, stage 5: Lossless’s fifth child, Multimedia (olive) – the one we left empty back in [figs. 5](#) and [15](#) – finally expands into the three media types we keep coming back to: video, image, and audio. Lossy is still collapsed on purpose; we expand it in [Part 9](#).

Let’s take each one in turn. Basically each of FFV1, PNG, and FLAC uses prediction to guess the current pixel or sample based on previously processed pixels or samples, then applies entropy coding to the residual error.

Lossless video – FFV1. FFV1 predicts each pixel from neighboring pixels that have already been

processed, then entropy-codes the prediction error with range coding. It's the format film archives and broadcasters reach for when a frame absolutely cannot change, not even by one pixel.

Lossless images – PNG. PNG runs a per-row prediction filter to exploit how similar neighboring pixels usually are, then compresses the resulting residual with DEFLATE, the same LZ77-plus-Huffman pipeline.

Lossless audio – FLAC. FLAC predicts each audio sample from the samples just before it, then entropy-codes the small prediction error with fast, simple Rice coding, the specialized prefix code we met back in [Part 4](#).

So if you guessed "a prediction step followed by entropy coding" for FLAC, you were right. That same pattern – predict, then entropy-code the error – shows up in FFV1 and PNG too. Multimedia isn't a new way of compressing data invented from scratch; it's the first four families, specialized for pixels and samples. That's exactly why it earns its own branch in the tree: not a new technique, but the place where the other four meet a specific kind of data.

Part 9 — Filling in lossy: video, images, and audio

Now let's finally fill in the other half of our master tree: lossy compression. The key takeaway of lossy video, image, and audio compression is that they discard high-frequency detail that is not easily perceptible to human eyes and ears. Thus, the reduction in quality significantly reduces the size of multimedia files, but it remains acceptable for human perception.

Unlike lossless, lossy compression doesn't reduce neatly into four shared families. Instead, it splits naturally by media type, because what counts as "unnoticeable" is different for your eyes than it is for your ears.

Lossy images – JPEG and WebP. JPEG splits the image into small blocks, transforms each block into frequency components with the discrete cosine transform, and then throws away the high-frequency detail the human eye is least likely to notice. WebP improves on that idea by borrowing prediction tricks from video compression, predicting each block from its neighbors before transforming it, which typically gets noticeably smaller files than JPEG at the same visual quality.

Lossy audio – MP3, AAC, and Opus. All three lean on a psychoacoustic model, a model of what the human ear can and can't actually perceive, to discard sound components you wouldn't hear anyway, then entropy-code whatever's left. MP3 is the original of the three, and the least efficient. AAC is its successor, using a finer-grained psychoacoustic model to match MP3's quality at a noticeably lower bitrate. Opus is the newest of the three, built for real-time use, and it switches between a speech-optimized mode and a music-optimized mode internally to stay efficient at both.

Lossy video – H.264, HEVC, and AV1. Video reuses the image trick at its core: each frame still gets the block-transform-and-discard treatment. But video adds one more move. It predicts each frame from neighboring frames using motion estimation, so it only has to encode what actually *changed* between frames, not the whole picture over and over. H.264 is the workhorse behind most streaming and Blu-ray video in use today. HEVC, also called H.265, refines the same idea with larger, more flexible prediction blocks and roughly doubles H.264's compression ratio at the same quality. AV1 is the newest of the three and, unlike the other two, is royalty-free; it generally edges out HEVC on ratio and is the format YouTube and Netflix increasingly default to for new streams.

Connect the dots: lossy video is really lossy image compression, applied frame by frame, plus one new idea. Can you name that new idea before reading the paragraph above again? (Hint: it's about what changes *between* frames, not within one.)

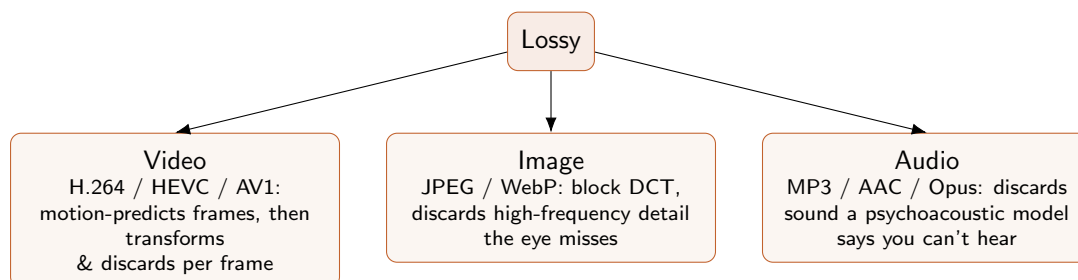


Figure 27: The lossy branch, fully grown. Video (left) builds directly on the same per-frame transform trick used for still images (middle), then adds motion prediction across frames. Audio (right) follows the same overall pattern – discard what perception can’t catch – using a hearing model instead of a vision model.

Part 10 — The complete tree

We’ve now filled in every branch we set out to cover. Let’s step back and look at the whole thing, lossless and lossy, side by side, one last time.

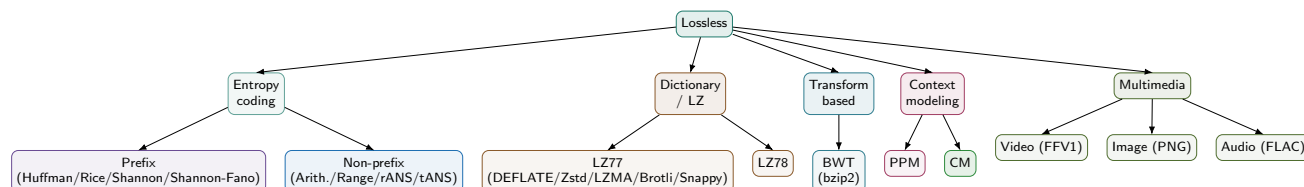


Figure 28: The complete tree, lossless half. Every colored leaf here is something we built up piece by piece over Parts 3 through 8.

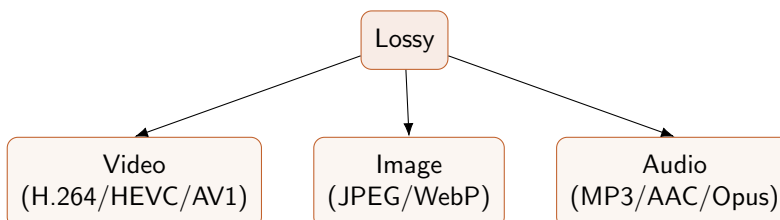


Figure 29: The complete tree, lossy half, from [Part 9](#). Lossy splits by medium instead of by algorithmic family, since what’s “unnoticeable” depends on whether you’re looking or listening.

And that’s the full map. Two root branches, lossless and lossy, five lossless families, and three media types under lossy. Every algorithm we study for the rest of this course slots into one leaf of this tree. Whenever you feel lost in a later video, come back to this picture and ask yourself: which leaf am I in right now?